

使用客戶自行管理的加密金鑰 (CMEK) 加密 Cloud Functions

來源：<https://codelabs.developers.google.com/use-cmek-to-encrypt-cloud-functions?hl=zh-tw>

步驟數：10

use-cmek-to-encrypt-cloud-functions

目錄

1. [1. 簡介](#)
2. [2. 設定和需求](#)
3. [3. 為 Cloud Functions 建立新的金鑰環和金鑰](#)
4. [4. 建立啟用 CMEK 的 Docker 格式 Artifact Registry 存放區](#)
5. [5. 授予服務帳戶金鑰存取權 \(第 2 代\)](#)
6. [6. 授予服務帳戶金鑰存取權 \(第 1 代\)](#)
7. [7. 建立以 CMEK 加密的函式 \(第 2 代\)](#)
8. [8. 建立以 CMEK 加密的函式 \(第 1 代\)](#)
9. [9. 叫用已停用加密金鑰的 CMEK 加密函式](#)
10. [10. 恭喜](#)

1. 簡介

總覽

[Cloud Functions](#) 是一項簡易運算解決方案，可讓開發人員建立獨立的單一用途函式來回應雲端事件，而不需要管理伺服器或執行階段環境。

您可以使用 [Cloud Key Management Service 客戶自行管理的加密金鑰 \(CMEK\)](#)，保護 Cloud Functions 和相關靜態資料。使用 CMEK 部署函式時，系統會使用您完全掌控的加密金鑰，保護與函式相關聯的資料。這類加密方式可協助您在金融服務等特定產業中，符合法規遵循要求。由於金鑰是您擁有，且不受 Google 控制，因此金鑰停用或毀損時，任何人都無法存取受這些加密金鑰保護的資料 (包括您)。

對於 Cloud Functions，CMEK 會加密下列項目：

- 為了進行部署作業所上傳，並由 Google 儲存在 Cloud Storage 中，用於建構程序的函式原始碼。
- 函式建構程序的結果，包括從函式原始碼建構的容器映像檔，以及已部署的每個函式執行個體。
- 內部事件傳輸管道的靜態資料 (僅限第 1 代)。

如要進一步瞭解加密資料，請參閱 [Cloud Functions CMEK 說明文件](#)。

建構項目

本程式碼實驗室說明如何部署使用 CMEK 加密的 Cloud Function (第 1 代或第 2 代)。本程式碼實驗室會使用公開 Cloud Function (也就是不需要驗證的函式) 進行示範。您可以叫用已啟用 CMEK 的已驗證函式，就像叫用任何其他需要驗證的 Cloud Functions 一樣。

注意：目前 Cloud Functions 第 2 代的 CMEK 功能為公開預先發布版。本程式碼實驗室會使用 [gcloud beta](#) 中的指令，說明 Cloud Functions 第 2 代的範例。

課程內容

- 如何在現有的對稱金鑰環上建立 CMEK 金鑰
 - 如何建立 Artifact Registry 存放區
 - 如何為第 1 代和第 2 代 Cloud Functions 設定 CMEK
-

2. 設定和需求

必要條件

- 您已登入 Cloud Console
- 您先前已部署 HTTP 觸發的 Cloud Functions (確認您具備適當的角色並已啟用 API)

啟用 Cloud Shell

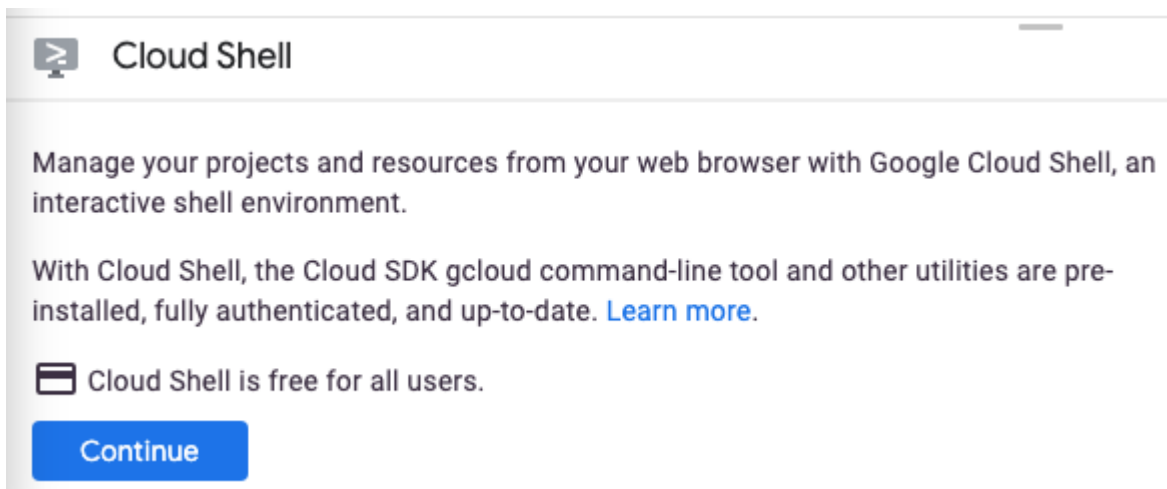
1. 在 Cloud 控制台，點選「啟用 Cloud Shell」圖示



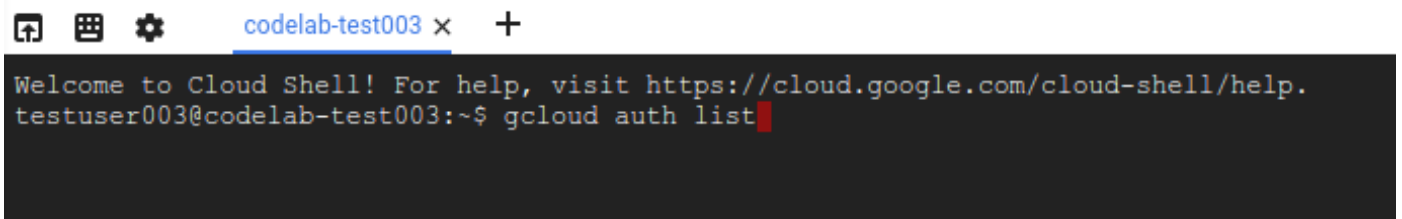
。



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面，說明這個指令列環境。如果出現中繼畫面，請按一下「繼續」。



佈建並連至 Cloud Shell 預計只需要幾分鐘。



這部虛擬機器已載入所有必要的開發工具，並提供永久的 5 GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器上完成。

連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID。

2. 在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

指令輸出

```
Credentialed Accounts
ACTIVE  ACCOUNT
*       <my_account>@<my_domain.com>

To set the active account, run:
$ gcloud config set account `ACCOUNT`
```

注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。詳情請參閱 [gcloud 指令列工具總覽](#)。

3. 在 Cloud Shell 中執行下列指令，確認 `gcloud` 指令知道您的專案：

```
gcloud config list project
```

指令輸出

```
[core]
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

注意： 如果您在 Cloud Shell 以外的環境完成本教學課程，請按照「[設定應用程式預設憑證](#)」一文的說明操作。

步驟 3 · 約 3 分鐘

3. 為 Cloud Functions 建立新的金鑰環和金鑰

執行下列指令，確認已啟用 Cloud KMS API：

```
gcloud services enable cloudkms.googleapis.com
```

首先，請建立環境變數，內含金鑰環名稱、金鑰名稱、區域，以及本程式碼研究室中使用的其他變數。

```
KEYRING_NAME="keyring-functions"
REGION="us-central1"
KEY_NAME="key-encrypted-function"
PROJECT_ID=$(gcloud config get-value project)
PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --
format='value(projectNumber)')
USER_EMAIL=$(gcloud config list account --format "value(core.account)")
```

接著，建立「金鑰環」，這是 Cloud KMS 金鑰和金鑰版本的根資源。

```
gcloud kms keyrings create $KEYRING_NAME --location $REGION
```

最後，您可以在 Cloud KMS 中，於新的金鑰環內建立[對稱金鑰](#)。

```
gcloud kms keys create $KEY_NAME --keyring $KEYRING_NAME --location $REGION --purpose
"encryption"
```

步驟 4 · 約 5 分鐘

4. 建立啟用 CMEK 的 Docker 格式 Artifact Registry 存放區

在本節中，您將在[啟用 CMEK 的 Artifact Registry](#) 中建立 Docker 格式的存放區。這組金鑰與您部署 Cloud 函式時使用的金鑰相同。

首先，您需要 Artifact Registry 的服務帳戶。執行下列指令即可建立：

```
gcloud beta services identity create --service=artifactregistry.googleapis.com --
project=$PROJECT_ID
```

使用下列指令將 [CryptoKey Encrypter/Decrypter IAM 角色](#) (`roles/cloudkms.cryptoKeyEncrypterDecrypter`) 授予 Artifact Registry 服務帳戶，取得金鑰的權限：

```
gcloud kms keys add-iam-policy-binding \
  $KEY_NAME --location $REGION --keyring=$KEYRING_NAME \
  --member serviceAccount:service-$PROJECT_NUMBER@gcp-sa-
artifactregistry.iam.gserviceaccount.com \
  --role roles/cloudkms.cryptoKeyEncrypterDecrypter
```

並將角色授予將在 Artifact Registry 中建立存放區的主體，例如您目前使用的有效帳戶。您可以執行 `gcloud auth list`，確認目前使用的帳戶。

```
gcloud kms keys add-iam-policy-binding \
  $KEY_NAME --location $REGION --keyring=$KEYRING_NAME \
  --member user:$USER_EMAIL \
  --role roles/cloudkms.cryptoKeyEncrypterDecrypter
```

您現在可以建立啟用 CMEK 的 Docker 格式存放區。

注意：區域必須與 CMEK 金鑰位於相同區域。

```
REPO_NAME=my-cmek-encrypted-repo

KEY_FULLPATH=projects/"$PROJECT_ID"/locations/"$REGION"/keyRings/"$KEYRING_NAME"/cryp
toKeys/"$KEY_NAME"

gcloud artifacts repositories create $REPO_NAME \
  --repository-format=docker \
  --location=$REGION \
  --kms-key=$KEY_FULLPATH \
  --async
```

執行下列指令，即可查看新的 Artifact Registry 存放區：

```
gcloud artifacts repositories describe $REPO_NAME --location=$REGION
```

5. 授予服務帳戶金鑰存取權 (第 2 代)

本節說明如何為第 2 代函式建立服務帳戶。如要建立第 1 代函式，請前往下一節。

您必須授予多個服務代理金鑰存取權，方法是授予 [CryptoKey Encrypter/Decrypter IAM 角色](#) (`roles/cloudkms.cryptoKeyEncrypterDecrypter`)。這些服務代理用於取得 Cloud Storage 中儲存的原始碼存取權、在 Artifact Registry 中受 CMEK 保護的儲存庫儲存函式映像檔，以及部署 CMEK 加密的 Cloud 函式。

注意 (如果您使用新專案)： 建立專案初期無法使用部分服務代理，例如 Cloud Storage 服務代理。這些服務代理會在您首次存取時啟用。您必須先啟用服務代理，才能為其指派權限。

如要啟用 Cloud Storage 服務代理，請擷取其電子郵件地址：

```
gcloud storage service-agent --project=$PROJECT_ID
```

第 2 代函式的步驟

1. 授予 Cloud Run 服務代理金鑰存取權：

```
CLOUDRUN_SA=service-$PROJECT_NUMBER@serverless-robot-prod.iam.gserviceaccount.com
```

```
gcloud kms keys add-iam-policy-binding $KEY_NAME \
--keyring=$KEYRING_NAME \
--location=$REGION \
--member=serviceAccount:$CLOUDRUN_SA \
--role=roles/cloudkms.cryptoKeyEncrypterDecrypter
```

2. 授予 Eventarc 服務代理金鑰存取權：

```
EVENTARC_SA=service-$PROJECT_NUMBER@gcp-sa-eventarc.iam.gserviceaccount.com
```

```
gcloud kms keys add-iam-policy-binding $KEY_NAME \
--keyring=$KEYRING_NAME \
--location=$REGION \
--member=serviceAccount:$EVENTARC_SA \
--role=roles/cloudkms.cryptoKeyEncrypterDecrypter
```

3. 授予 Artifact Registry 服務代理人金鑰存取權：

```
AR_SA=service-$PROJECT_NUMBER@gcp-sa-artifactregistry.iam.gserviceaccount.com
```

```
gcloud kms keys add-iam-policy-binding $KEY_NAME \
--keyring=$KEYRING_NAME \
--location=$REGION \
--member=serviceAccount:$AR_SA \
--role=roles/cloudkms.cryptoKeyEncrypterDecrypter
```

4. 授予 Cloud Storage 服務代理金鑰存取權：

```
STORAGE_SA=service-${PROJECT_NUMBER}@gs-project-accounts.iam.gserviceaccount.com
```

```
gcloud kms keys add-iam-policy-binding $KEY_NAME \  
--keyring=$KEYRING_NAME \  
--location=$REGION \  
--member=serviceAccount:$STORAGE_SA \  
--role=roles/cloudkms.cryptoKeyEncrypterDecrypter
```

下一節將說明如何建立及部署 CMEK 加密函式。

6. 授予服務帳戶金鑰存取權 (第 1 代)

本節說明如何為第 1 代函式建立服務帳戶。如果您先前已為第 2 代函式建立服務帳戶，請前往下一節。

您必須授予多個服務代理金鑰存取權，方法是授予 [CryptoKey Encrypter/Decrypter IAM 角色](#) (`roles/cloudkms.cryptoKeyEncrypterDecrypter`)。這些服務代理用於取得 Cloud Storage 中儲存的原始碼存取權、在 Artifact Registry 中受 CMEK 保護的儲存庫儲存函式映像檔，以及部署 CMEK 加密的 Cloud 函式。

注意 (如果您使用新專案)： 建立專案初期無法使用部分服務代理，例如 Cloud Storage 服務代理。這些服務代理會在您首次存取時啟用。您必須先啟用服務代理，才能為其指派權限。

如要啟用 Cloud Storage 服務代理，請擷取其電子郵件地址：

```
gcloud storage service-agent --project=$PROJECT_ID
```

第 1 代函式的步驟

1. 授予 Cloud Functions 服務代理程式金鑰存取權：

```
FUNCTION_SA=service-$PROJECT_NUMBER@gcf-admin-robot.iam.gserviceaccount.com
```

```
gcloud kms keys add-iam-policy-binding $KEY_NAME \
--keyring=$KEYRING_NAME \
--location=$REGION \
--member=serviceAccount:$FUNCTION_SA \
--role=roles/cloudkms.cryptoKeyEncrypterDecrypter
```

2. 授予 Artifact Registry 服務代理人金鑰存取權：

```
AR_SA=service-$PROJECT_NUMBER@gcp-sa-artifactregistry.iam.gserviceaccount.com
```

```
gcloud kms keys add-iam-policy-binding $KEY_NAME \
--keyring=$KEYRING_NAME \
--location=$REGION \
--member=serviceAccount:$AR_SA \
--role=roles/cloudkms.cryptoKeyEncrypterDecrypter
```

3. 授予 Cloud Storage 服務代理金鑰存取權：

```
STORAGE_SA=service-$PROJECT_NUMBER@gs-project-accounts.iam.gserviceaccount.com
```

```
gcloud kms keys add-iam-policy-binding $KEY_NAME \
--keyring=$KEYRING_NAME \
--location=$REGION \
--member=serviceAccount:$STORAGE_SA \
--role=roles/cloudkms.cryptoKeyEncrypterDecrypter
```

下一節將說明如何建立及部署 CMEK 加密函式。

步驟 7 · 約 0 分鐘

7. 建立以 CMEK 加密的函式 (第 2 代)

本節說明如何建立第 2 代函式。如需第 1 代的相關說明，請前往下一節。

您已設定 Artifact Registry 存放區並啟用 CMEK，也授予 Cloud Functions 金鑰存取權，現在可以部署使用 CMEK 金鑰加密的函式。

第 2 代函式的步驟：

建立函式的原始碼

雖然本程式碼研究室使用 Node.js，但您可以使用任何[支援的執行階段](#)。

首先，請建立目錄並 cd 到該目錄。

```
mkdir ~/cmek-function-2ndgen && cd $_
```

接著，建立 package.json 檔案。

```
touch package.json

echo '{
  "dependencies": {
    "@google-cloud/functions-framework": "^2.1.0"
  }
}' > package.json
```

接著，建立 index.js 來源檔案。

```
touch index.js

echo 'const functions = require("@google-cloud/functions-framework");

functions.http("helloWorld", (req, res) => {
  res.send(`Hello ${req.query.name || req.body.name || "World"}!`);
});' > index.js
```

使用 CMEK 加密功能部署第 2 代 Cloud Functions

注意：以下範例說明如何使用目前目錄中的來源部署函式。確認您與函式的原始碼位於同一目錄。

```
FUNCTION_NAME=protect-me-cmek-2ndgen
ENTRY_POINT=helloWorld

REPO_FULLPATH=projects/"$PROJECT_ID"/locations/"$REGION"/repositories/$REPO_NAME

gcloud beta functions deploy $FUNCTION_NAME \
--gen2 \
--region $REGION \
--kms-key $KEY_FULLPATH \
--docker-repository $REPO_FULLPATH \
--source . \
--trigger-http \
--allow-unauthenticated \
--runtime nodejs16 \
--entry-point $ENTRY_POINT
```

執行這項指令，即可在輸出結果中查看 CMEK 金鑰

```
gcloud functions describe $FUNCTION_NAME --region $REGION | grep kmsKeyName
```

測試第 2 代函式

您可以透過 curl 測試函式：

```
FUNCTION_URL="$(gcloud functions describe $FUNCTION_NAME --region $REGION --
format='get(serviceConfig.uri)')"
```

```
curl $FUNCTION_URL
```

這會產生下列結果：

```
Hello World!
```

只要啟用加密金鑰，函式就會向呼叫端傳回成功訊息。不過，一旦停用加密金鑰，來電者就會收到錯誤訊息。

在下一節中，您會看到停用金鑰後叫用函式時會發生什麼情況。

8. 建立以 CMEK 加密的函式 (第 1 代)

本節說明如何建立第 1 代函式。如果您先前已建立第 2 代函式，請前往下一節。

您已設定 Artifact Registry 存放區並啟用 CMEK，也授予 Cloud Functions 金鑰存取權，現在可以部署使用 CMEK 金鑰加密的函式。

第 1 代函式的步驟如下：

建立第 1 代函式的原始碼

雖然本程式碼研究室使用 Node.js，但您可以使用任何[支援的執行階段](#)。

首先，請建立目錄並 cd 到該目錄。

```
mkdir ~/cmek-function-1stgen && cd $_
```

接著，建立 package.json 檔案。

```
touch package.json

echo '{
  "name": "function-cmek-codelab",
  "version": "0.0.1"
}' > package.json
```

接著，建立 index.js 來源檔案。

```
touch index.js

echo "exports.helloWorld = (req, res) => {
  let message = req.query.message || req.body.message || 'Hello World!';
  res.status(200).send(message);
};" > index.js
```

使用 CMEK 加密部署第 1 代 Cloud Functions

注意：以下範例說明如何使用目前目錄中的來源部署函式。確認您與函式的原始碼位於同一目錄。

```
FUNCTION_NAME=protect-me-cmek-1stgen
ENTRY_POINT=helloWorld

REPO_FULLPATH=projects/"$PROJECT_ID"/locations/"$REGION"/repositories/$REPO_NAME

gcloud functions deploy $FUNCTION_NAME \
--region $REGION \
--kms-key $KEY_FULLPATH \
--docker-repository $REPO_FULLPATH \
--source . \
--trigger-http \
--allow-unauthenticated \
--runtime nodejs16 \
--entry-point $ENTRY_POINT
```

執行這項指令，即可在輸出結果中查看 CMEK 金鑰

```
gcloud functions describe $FUNCTION_NAME --region $REGION | grep kmsKeyName
```

測試第 1 代函式

您可以透過 curl 測試函式：

```
FUNCTION_URL="$(gcloud functions describe $FUNCTION_NAME --region $REGION --
format='get(httpsTrigger.url)')"
```

```
curl $FUNCTION_URL
```

這會產生下列結果：

```
Hello World!
```

只要啟用加密金鑰，函式就會向呼叫端傳回成功訊息。不過，一旦停用加密金鑰，來電者就會收到錯誤訊息。

在下一節中，您會看到停用金鑰後叫用函式時會發生什麼情況。

9. 叫用已停用加密金鑰的 CMEK 加密函式

在最後一節中，您將使金鑰失效，並再次叫用函式，查看產生的錯誤。

停用加密金鑰

您可以執行這項指令 [停用金鑰](#)。由於本程式碼研究室只會建立一個金鑰版本，因此您將停用版本 1。

```
gcloud kms keys versions disable 1 \
  --key=$KEY_NAME \
  --keyring=$KEYRING_NAME \
  --location=$REGION
```

您應該會看到以下資訊：

```
algorithm: GOOGLE_SYMMETRIC_ENCRYPTION
createTime: '2023-04-11T03:30:49.111832653Z'
generateTime: '2023-04-11T03:30:49.111832653Z'
name: projects/dogfood-gcf-saraford/locations/us-central1/keyRings/myKeyRing/cryptoKeys/encrypted-function/cryptoKeyVersions/1
protectionLevel: SOFTWARE
state: DISABLED
```

注意： 變更需要幾小時才會生效。

使用已停用的金鑰叫用函式

現在再次執行 `curl` 函式。

```
curl $FUNCTION_URL
```

這次不會收到 Hello World 回覆。

在 Cloud Function 的記錄中，您會看到

```
User's CMEK key has been disabled. CMEK key: projects/<PROJECT-NAME>/locations/us-central1/keyRings/myKeyRing/cryptoKeys/encrypted-function
```

嘗試在停用 CMEK 金鑰時查看資源

本節會說明停用 CMEK 金鑰後，下列資源會無法使用：

- 函式原始碼
- 從原始碼建構的容器映像檔

舉例來說，如果前往 Cloud Function 的「來源」分頁，系統會顯示擷取封存檔時發生錯誤。如果您嘗試直接在 Cloud Storage 中查看含有原始碼的 .zip 檔案，也會收到類似的錯誤訊息。

METRICS

DETAILS

SOURCE

VARIABLES

Runtime : Node.js 18

Entry point : helloWorld



Unknown error while fetching the archive

此外，您也無法從 Artifact Registry 存取函式的容器映像檔。舉例來說，如果您嘗試將該容器映像檔部署至 Cloud Run，系統會顯示找不到映像檔的錯誤訊息。

如需加密資源的完整清單，請參閱 CMEK 函式說明文件。

步驟 10 · 約 1 分鐘

10. 恭喜

恭喜，您已完成本程式碼研究室！

涵蓋內容

- 如何在現有的對稱金鑰環上建立 CMEK 金鑰
- 如何建立 Artifact Registry 存放區
- 如何為 Cloud Function 設定 CMEK

詳情請參閱

如要進一步瞭解 Cloud Functions 和 CMEK，請參閱下列連結：

- [使用 CMEK 保護 Cloud Functions](#) 一文
 - [客戶管理的加密金鑰說明文件](#)
-